1 Define Requirements

Start by clearly identifying what your robot hand needs to do.

- Functionality: Grasp objects? Perform delicate tasks? Mimic a human hand?
- Degrees of Freedom (DoF): Full 5-fingered hand or simplified 2-3 finger gripper?
- Payload: How much weight must it lift?
- Size constraints: Human-sized or scaled differently?
- Environment: Indoor? Harsh/industrial environment?

Design Principles:

- User-Centered Design: Tailor the hand's capabilities to the task/user needs.
- Feasibility and Constraints: Balance ideal goals with available resources (budget, tools, time).
- Functional Decomposition: Break down the hand's tasks (e.g., grasping, lifting, sensing).

Maker notes:

- I wanted to grasp a cylindrical tube on my desk. It's made of cardboard but quite sturdy. It's lightweight and has a diameter of about 28mm. I could go for a 2-finger gripper with an exactly shaped fingertip to grab it with vice like grip, but I wanted to grab it more human like.
- To grasp it firmly, I wanted to grab the object with 2 fingers and a thumb to hold it securely.
- The object weighs just 79g
- It's 28mm diameter and 310mm long. a gripper akin to the size of a human hand would work well.
- Grabbing the indoors in an industrial environment in close proximity to humans.
- I just need the fingers to be able to curl around the object, which is why I designed each finger to have 3 links (prox, mid and distal).
- As the object is lightweight and small, I make use cost-efficient materials and electronics.
- Functionally, to grab the object I just need the hand to open and close.

2 Conceptual Design

Sketch and plan the basic architecture.

- Finger count and joints: How many fingers and how many joints per finger?
- Actuation method: Tendon-driven, servo-based, pneumatic, etc.
- Materials: PLA (3D printing), aluminum, carbon fiber, silicone, etc.
- Grip type: Parallel, adaptive, underactuated?
- Tools: Pen & paper, Sketchbook, or CAD software (e.g., Fusion 360, SolidWorks).

Design Principles:

- Simplicity & Modularity: Favor simple, modular designs to make building, testing, and iteration easier.

- Form Follows Function: Let the hand's purpose dictate its shape and mechanics.
- Design for Assembly (DFA): Plan joints and parts that are easy to assemble and disassemble.

Make notes:

- I intended to make 3 fingers with 3 joints each.
- Each finger driven by a single tendon, the shape of the object allows for this and I'm not planning on the same hand to grab other shaped objects.
- Using PLA as the material is sufficiently strong in the sense that the motor won't break it and the torque exerted from the finger won't break it or damage the cardboard cylindrical tube.
- Underactuated works as the finger can conform to the cylindrical shape passively, and this simplifies the design.
- I'm using SolidWorks.
- I wish to use minimal parts and use easy-to-access off-the-shelf components.
- I'm almost replicating how our own fingers work.
- In the design I am using stiff wire as the tendons. So I must accommodate for the wires bending radius in the design. I have also minimised the parts needed to assemble it, opting for spring steel pins in place of threaded inserts and bolts. I have also considered how the bearings can be placed. From the outside as these need to be pressed into place with a arbour press.

3 CAD Modelling

Build a 3D model of your robot hand.

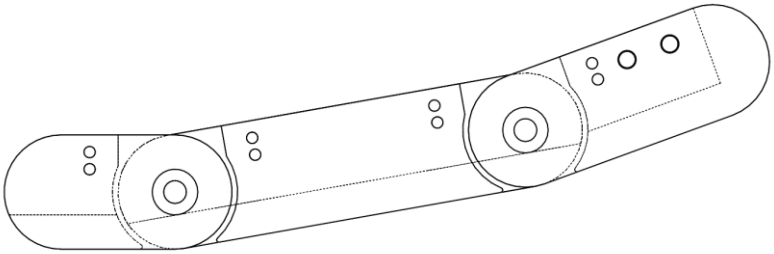
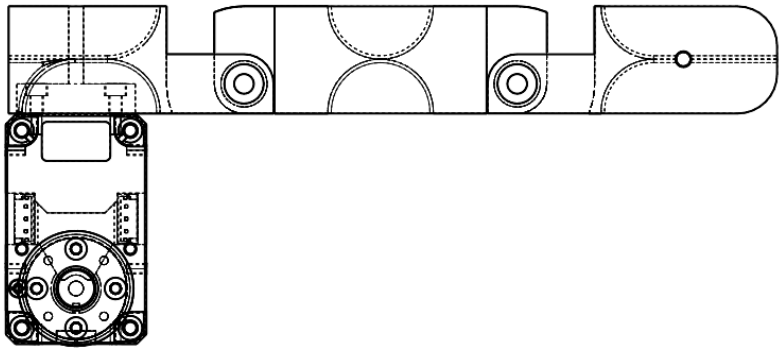
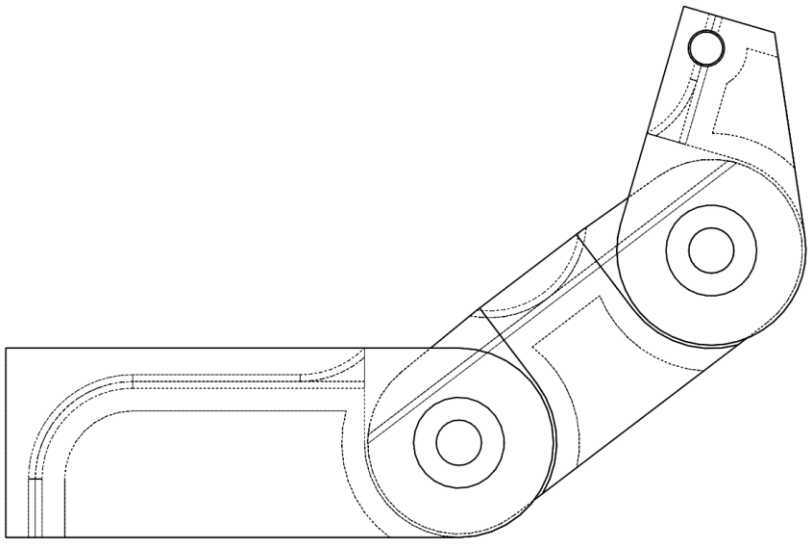
- Use parametric CAD software like Fusion 360 or SolidWorks.
- Design the palm, fingers, and joint mechanisms.
- Plan space for actuators, wiring, and sensors.

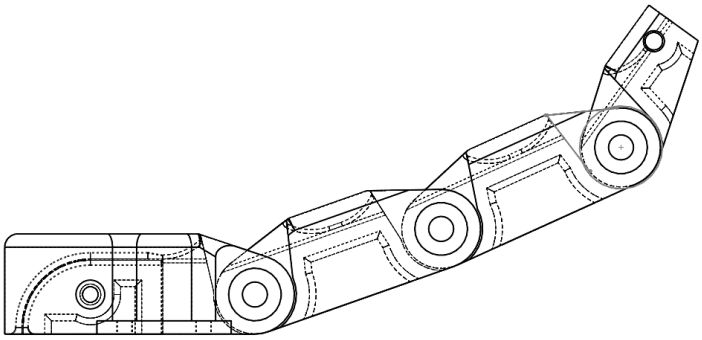
Design Principles:

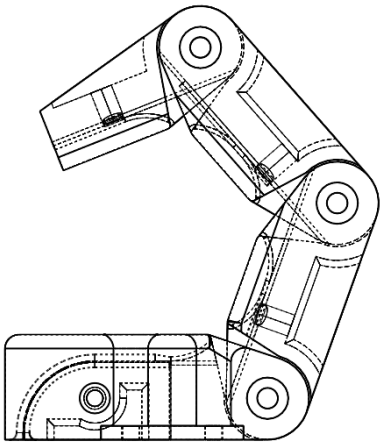
- Parametric Design: Use adjustable parameters for fast iterations.
- Tolerance Awareness: Model realistic clearances for manufacturing and fit.
- Stress and Load Analysis: Estimate forces to prevent mechanical failure.

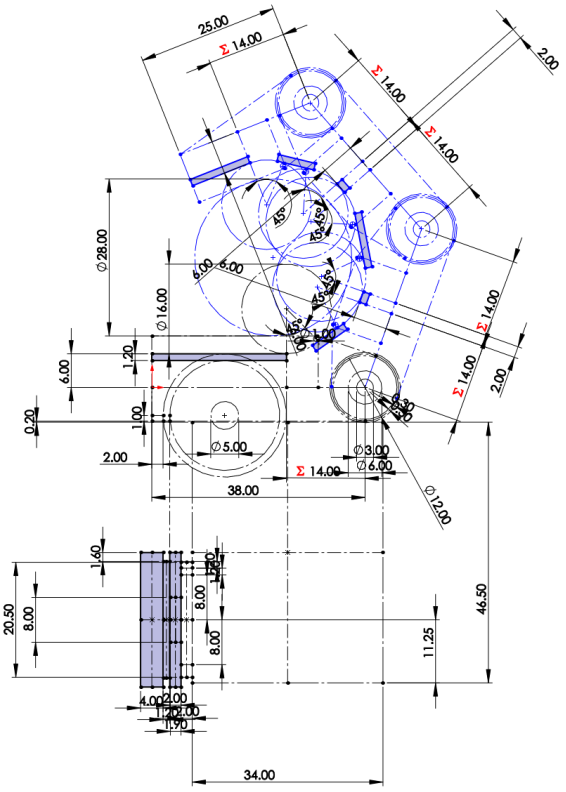
Maker notes:

- I went through several iterations of the design:
    - Changing the size of bearings
    - How the finger links were joined
    - The tendon routing
    - The link lengths
    - The tolerances for the bearing fits
    - The tolerances between each link
    - The tolerances for the tendon
    - The wall thickness of the plastic for strength and also to accommodate the 3D printers limitations. (if the 3D printer nozzle is 0.4mm, I design any thin walls to be a factor of it, so 1.2mm walls is 3 nozzles thick for example.

    - The orientation the parts are to be 3D printed
    - Had to make sure each finger can't collide with its neighbour
- My designs allows me to change parameters such as:
    - Link lengths
    - Tendon bend radius
    - Tolerances
    - Wall thicknesses
    - Joint angle limits
    - Bearing sizes.
    - Tendon diameter
- 
- 
- 







4 Actuation Design

Choose how to move the fingers.

- Servos: Easy to control, common in hobby robotics.
- DC motors + gearboxes: Good for strength, needs more control circuitry.
- Tendon or cable-driven systems: Mimic human hand.
- Pneumatics or hydraulics: For soft robotics or high-force needs.

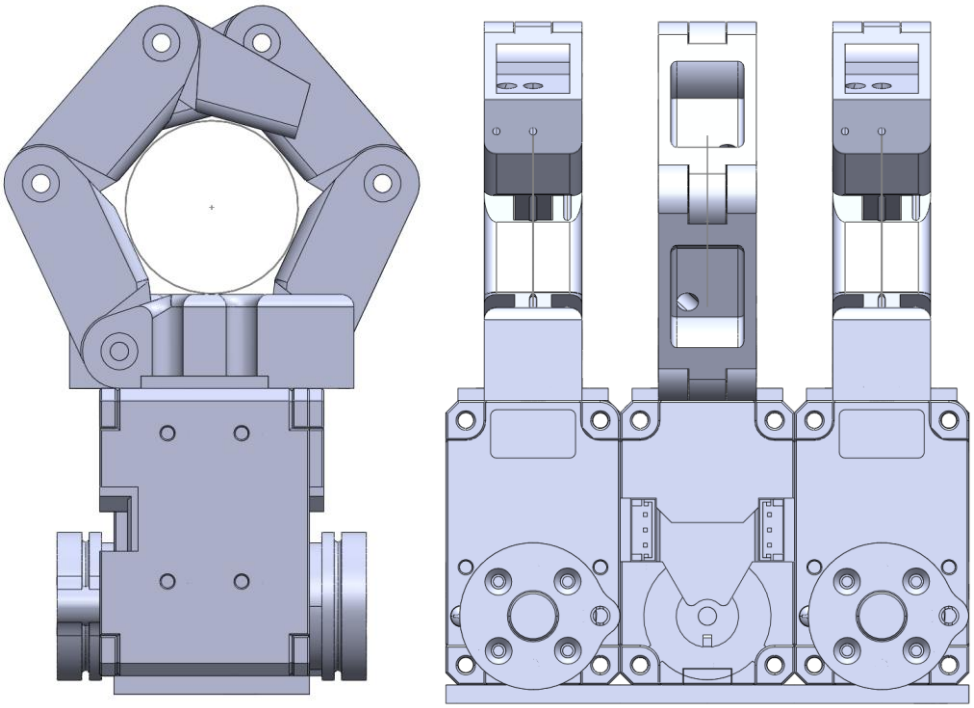
Also decide:

- How many motors? One per joint or underactuated design?

Design Principles:

- Efficiency: Choose actuators that match torque/speed needs without waste.
- Redundancy Minimization: Avoid excessive motors; underactuated designs are often better.
- Mechanical Advantage: Use gearing or linkages to optimize force and speed.

Maker notes:

- I'm a big fan of Dynamixels by Robotis. They're servo gear motors with set sizes and gear ratios and easy to control.
- Went for tendon driven, direct driven joints would have been huge and wasteful.
- For the task a Dynamixel XL430-W250-T is enough, if I need more torque or current-position control, I can swap over to a XM430-W350-T which has the same form factor. Just costs more.
- At the cost of chunkier fingers, I can increase the mechanical advantage by increasing the distance between the tendon and the pivot points.
- 

5 Electronics & Sensors

Decide on the control system and feedback.

- Microcontroller: Arduino, Teensy, or a Raspberry Pi for more complex control.
- Motor drivers: For servo or DC motor control.

Sensors:

- Force/pressure sensors (e.g., capacitive, resistive).

- Flex sensors for joint feedback.
- IMUs or encoders.
- Power supply: Battery or wired?

Design Principles:

- Separation of Concerns: Keep sensing, actuation, and logic circuits modular and separate.
- Noise Reduction: Ensure signal integrity, especially for analog sensors.
- Fail-Safe Design: Include limits or sensors to prevent hardware damage.

Maker notes:

- For our use case, I'm controlling using a laptop
- The beauty of Dynamixels is they have a built in motor driver.
- The beauty of Dynamixels is they tell me values such as current draw, position, motor load, voltage. I can infer how strong each finger is grasping the object with these.
- The system does not need to be wireless or portable so it is wired for power.

6 Fabrication

Build the hand.

- 3D printing for prototyping parts.
- Laser cutting/CNC if needed for stronger parts.
- Assemble everything with appropriate fasteners, bearings, etc.

Design Principles:

- Design for Manufacturability (DFM): Optimize parts for the tools and materials you have.
- Material Selection: Match materials to stress, wear, and flexibility needs.
- Tolerance Stacking: Be mindful of small misalignments adding up in assemblies.

Maker notes:

- All 3D printed and assembled by hand.
- Only purchased parts were:
    - Tendon wire
    - Dynamixel motor
    - Bearings
    - Steel pins
    - Threaded insert M2
    - M2 bolts
- Made sure the printed parts were easy to clean up and had the right tolerances.

7 Software & Control

Develop code to control the hand.

- Firmware: Low-level control (e.g., Arduino code for servos).

- Kinematics: Forward/inverse kinematics for finger motion.
- Grip strategies: Pre-programmed grips, force feedback control.
- Optional AI: Object recognition or grasp planning.

Languages: C++, Python (ROS if needed).

Design Principles:

- Layered Architecture: Divide software into layers (low-level motor control, mid-level motion planning, high-level logic).
- Real-Time Responsiveness: Ensure control loops are fast enough for smooth, stable movement.
- Scalability: Write modular, readable code to support future upgrades or added features.

Maker notes:

- I'm using python 3 for ease
- I want each finger to close at the same time.
- As each motor has a built-in encoder, I know the absolute position the motor needs to be at for the finger to be open and closed to hold the object.
- I have fine control over the motor to loosen or tighten the grip by altering the motor position (how much it pulled the tendon).
- I could, in theory, close the fingers and have it monitor the motor load. So it will grasp the object with the same amount of effort, even if the object diameter changes slightly.

8 Testing & Iteration

Test in stages:

- Finger movement.
- Full hand motion.
- Object interaction.

Collect feedback, find weak points, and iterate on the design.

Design Principles:

- Iterative Prototyping: Test early and often — fail fast to learn quickly.
- Data-Driven Design: Use measurable feedback (force, accuracy, speed) to refine the design.
- Root Cause Analysis: Fix problems at their source, not just the symptoms.

Maker notes:

- I manufactured a single finger of each iteration design, testing for:
    - tendon bending stiffness
    - tolerances
    - manufacturability
- I then replicated the final design twice and assembled the hand.

9 Documentation & Presentation

Document the process for future improvements or team collaboration.

- CAD files and schematics
- Wiring diagrams
- Software repo (e.g., GitHub)
- Photos/videos of testing

Design Principles:

- Reproducibility: Include enough detail for someone else to replicate your work.
- Clarity & Simplicity: Use diagrams, short explanations, and visuals.
- Version Control: Use tools like Git to track changes in code and design.

10Optional Enhancements

- Soft skin or compliant materials.
- Tactile sensing for feedback.
- Voice/gesture control integration.
- ROS integration if we want it working with a robot system.

Design Principles:

- Adaptability: Design to easily swap sensors or fingers.
- User Interaction: Consider ergonomics or intuitive control if the hand is meant to be teleoperated.
- Bio-Inspiration: Mimic biological hands for more natural and functional movement.

11 Images